Part 2: Indexing and Evaluation

Group members: Natàlia Grandas (u251664), Martina Frisach (u231662)
GitHub Repository: https://github.com/martinafrisach/IRWA-2026-Final-Project
Submission TAG: IRWA-2026-part-2

1. Overview

In Part 2 we built an inverted index over the pre-processed dataset, implemented conjunctive (AND) search with TF-IDF ranking, defined five test queries, and implemented seven evaluation metrics. We evaluated the engine on the two queries of validation_labels.csv and on five queries that we judged ourselves.

2. Part 2: Indexing

2.1. Fields and pre-processing

As we decided in Part 1, we use the hybrid approach, where title, description, brand, seller and product_details are in separate fields, and category and sub_category are merged into one category. For the product_details field we kept only the values and discarded the keys, since terms like fabric or color appear in almost every product and wouldn't give any important information. The numeric fields are not indexed as text, as decided in Part 1. Every text field and every query goes through the same build_terms function from Part 1 (lowercasing, punctuation removal, tokenization, stop-word removal and stemming), so that query terms match the stored terms.

2.2. Inverted index

Following the first class practice, we wrote the function create_index, which builds the inverted index for one field. For each document it saves the positions where each term appears and merges them into the main index. Therefore, the index[field][term] is a list of [doc_id, positions] pairs. In other words, the posting list of a term is the list of document ids, and the positions would give us the option to use phrase queries. Then using this function, we build one index per each field. In total, we have 738 terms for the title, 4,698 for the description, 40 for the category, 402 for the brand, 687 for the seller and 17,812 for the details. We chose separate indexes instead of one merged index so that each field can have a different weight in the ranking, even though it occupies more memory, since a term can appear in several indexes.

2.3. Conjunctive (AND) search

First, we pre-processed the query and repeated words were removed. For each query term we collect the documents that contain it in any field, and then we do the intersection of these sets. The final results include only documents that contain all the query terms, even if the words appear in different fields. For example, a document can match the query "women full sleeve sweatshirt cotton" if "women", "full", "sleeve" and "sweatshirt" appear in the title, while "cotton" appears in the product details. If a query term does not exist in the index, no documents are returned.

2.4. TF-IDF ranking

The documents that satisfy the AND condition are ranked by a TF-IDF score over the query terms and the fields. For each query term, e add up the weight of each field multiplied by 1 + log(tf), and multiply the result by idf = log(N / df), where tf is the number of times the term appears in that field, df is the number of documents containing the term in any field and N is 28,080, which is the total number of documents.. The logarithm makes repeated words help with diminishing returns, and the idf gives more weight to rare terms.

We chose the field weights based on two pieces of evidence: the validation labels results from Part 1 and the vocabulary size of each index.

First, the validation results showed that the title and product details contain approximately 80% of the query terms, while the description covers less terms. The category, brand and seller fields did not contain any query terms in the relevant documents.

Second, the size of each index would provide information on how useful each field is for distinguishing between products. The category has only 40 distinct terms, meaning that many products share the same categories. Therefore, it receives a low weight of 0.5. In contrast, the product details have the richest vocabulary, with 17,812 distinct terms, making matches in this field more informative. We therefore assign it a weight of 2.0.

The title receives the highest weight of 3.0 because, even though it has a small vocabulary of 738 terms, it provides a short and precise description of the product and covers most query terms. The description has a larger vocabulary of 4,698 terms, but it is longer and less specific, so it receives a lower weight of 1.5.

Finally, the brand and seller fields receive weights of 1.0 and 0.0. Brand names help identify relevant products, while the seller field describes who sells the product rather than what the product is. However, since the validation queries did not contain brand or seller terms, we don't have much evidence for these two weights, so they could be changed later.

2.5. Test queries

We chose the queries from the most frequent terms of the index, ranked by number of postings across fields. Most of the words we use are among the 40 most frequent terms like *men, women, shirt, topwear, cotton, slim…* We combined them into a query to describe different aspects of a product. We added two garment types that are not among the 40 most frequent terms, so that the queries cover different kinds of products. The five queries and the number of documents they match are:

1. men round neck cotton shirt - 3227 documents
2. women printed topwear - 3820 documents
3. men slim fit jeans - 212 documents
4. women cotton dress - 517 documents
5. men solid polo shirt - 919 documents

They are different in size and purpose. Queries 1 and 2 match thousands of documents, so they test the ranking when many candidates qualify, and query 2 uses a category term (*topwear*), so it is the broadest one. Query 3 is narrow and close to the second validation label query, so it allows us to compare our execution with the provided labels. Query 4 combines a fabric with a garment type, and query 5 combines a pattern with a specific style of shirt, with a medium number of matches.

3. Part 2: Evaluation

3.1 Evaluation Metrics Implementation

In this part we have implemented the following metrics: Precision@K, Recall@K, Average Precision@K, F1-Score@K, Mean Average Precision, Mean Reciprocal Rank and Normilized Discounted Cumulative Gain.

Precision@K

Measures the proportion of relevant documents among the first K results.

$$P@K = \frac{\#\ of\ relevant\ docs\ in\ the\ first\ K}{K}$$

For computing this in the project, we pass the retrieved docs and then sum the 1 values in the top K. Finally, we divide into K (k is a parameter we pass to the function).

Recall@K

Measures how many of the relevant documents have been retrieved.

$$R@K = \frac{\#\ of\ relevant\ docs\ in\ the\ first\ K}{\#\ od\ relevant\ docs\ for\ the\ query}$$

To compute Recall@K, we pass the relevance labels and the total number of relevant documents to the function. We sum the 1 values among the top K labels, as in Precision@K and divide by the total number of relevant documents.

Average Precision @ K

It measures the quality of the ranking, giving the best results when the relevant documents appear in the first positions.

$$AP@K = \frac{\sum_{i=1}^{K} P@i \cdot rel(i)}{\sum_{i=1}^{K} rel(i)}$$

Where P@i is the precision computed up to position i.

To compute this, we iterate through the first K relevance labels. Each time we find a relevant document, we calculate the precision at this position and add it to the total. Finally, we compute the mean of all stored precision values.

F1-Score@K

It combines Precision@K and Recall@K through their harmonic mean.

$$F1@K = 2 \cdot \frac{P@K \cdot R@K}{P@K + R@K}$$

To compute this metric, we calculate the precision at k and recall at k of the retrieved labels and return the formula.

Mean Average Precision (MAP)

It measures the average ranking quality across multiple queries.

$$MAP@K = \frac{1}{Q}\sum_{q=1}^{Q} AP@K(q)$$

Where Q is the total number of evaluated queries and AP@K is the Average Precision@K for query q.

To compute MAP, we calculate the AP@K for each query and then compute the mean of these values. This gives us a single value representing the overall ranking performance of the search engine across the evaluated queries.

Reciprocal Rank@K and Mean Reciprocal Rank (MRR)

MRR measures how early the first relevant document appears in the retrieved results.

First we calculate the Reciprocal Rank for each query:

$$RR@k(q) = \frac{1}{rank_q}$$

Where rank(q) is the position of the first relevant document for query q among the first K results.

Then we calculate the MRR:

$$MRR = \frac{1}{Q}\sum_{q=1}^{Q} RR@K(q)$$

To compute MRR, we first use rr_at_k, passing the relevance labels and K- The function iterates over the first K documents and returns the reciprocal of the position of the first relevant document.

Then, we pass the list of Reciprocal Rank values for all queries (per_query_rr) to mrr_at_k, which calculates and returns their mean.

Normalized Discounted Cumulative Gain (NDCG)

Measures the quality of the ranking by giving more importance to relevant documents appearing in the first positions.

First we calculate the Discounted Cumulative Gain (DCG):

$$DCG@K = \sum_{i=1}^{K} \frac{2^{rel(i)}-1}{log_2(i+1)}$$

Then we normalize it using the ideal ranking:

$$NDCG@K = \frac{DCG@K}{IDCG@K}$$

Where rel(i) represents the relevance of the document at position i, and IDCG@K is the maximum possible DCG for the ideal ranking.

To compute NDCG@K we pas the relevance labels, K, ans optionally the total number of relevant documents.

First, the function dcg_at_k computes the gain of each document based on its relevance and divides it by the logarithmic discount. Then, ndcg_at_k calculates the ideal DCG. If n_relevant is passed, the ideal ranking contains as many relevant documents as possible within the first K positions. Otherwise, the function sorts the available relevance labels in descending order to obtain the ideal ranking.

Finally, we divide the actual DCG by the ideal DCG.

3.2 Evaluation on Predefined Queries

Metric	**Query 1**	**Query2**
Precision@10	1.000	0.600
Recall@10	0.769	0.600
Average Precision@10	1.000	0.915
F1-Score@10	0.870	0.600
Reciprocal Rank@10	1.000	1.000
NDCG@10	1.000	0.706

Overall metric	**Value**
MAP@10	0.958
MRR@10	1.000

3.3 Manual Evaluation and Discussion

To evaluate our search engine manually, we selected 5 different queries and examined the first 10 retrieved documents for each one. We assigned a binary relevance label to each document: 1 if the product was relevant to the query and 0 otherwise.

The five queries used were:

1. women round neck cotton shirt
2. women printed topwear

3. men slim fit jeans
4. women cotton dress
5. men solid polo shirt

For doing this, first we create a dataframe to store for each query, the documents retrieved with our search function, and their manual relevance judgement.

To make the manual relevance, we printed the titles and descriptions of each document for each query and reason if they were relevant or not.

For example, in the first query, the first retrieved document was not relevant since it was a V neck T-Shirt and not rounded.

Doing this process for each query, we obtain the following labels and metric results:

Query 1: men round neck cotton shirt

Relevance labels: [0, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Query 2: women printed topwear

Relevance labels: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Query 3: men slim fit jeans

Relevance labels: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Query 4: women cotton dress

Relevance labels: [0, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Query 5: men solid polo shirt

Relevance labels: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Metric	**Query 1**	**Query 2**	**Query 3**	**Query 4**	**Query 5**
Precision@10	0.900	1.000	1.000	0.900	1.000
Average Precision@10	0.786	1.000	1.000	0.786	1.000
Reciprocal Rank@10	0.500	1.000	1.000	0.500	1.000
NDCG@10	0.833	1.000	1.000	0.833	1.000

The overall MAP@10 was 0.914 and the MRR@10 was 0.800.

Discussion of the results

The results show that our search engine performs well for the 5 selected queries. For queries 2,3, and 5, all the first 10 retrieved documents were considered relevant, so the evaluation metrics reached their maximum values of 1.000.

For queries 1 and 4, 9 out of 10 were relevant. However, the first retrieved document was not relevant in either case. This explains why Precision@10 was 0.900 while Average Precision@10 and NDCG@10 were lower, since these metrics also consider the position of relevant documents in the ranking.

The Reciprocal Rank@10 was 0.500 for these 2 queries because the first relevant document appeared in the second position. This shows that different metrics evaluate different aspects of search quality.

Limitations and Possible improvements

Although the results are positive in general, our search engine still has some limitations.

First, the retrieval system uses an inverted index with conjunctive AND retrieval. This means that documents must contain all the query terms to be retrieved, which can exclude relevant products that use different words or synonyms.

Second, the ranking is based on TF-IDF, which considers the importance of query terms but does not fully understand their meaning. As a result, a document containing the query terms may appear before a more relevant product.

Another limitation is that, although we use different weights for each product field, these weights were selected manually and may not be optimal for every query. Also, the results depend on how the user formulates the query, since different words or spelling variations may affect which documents are retrieved.

To improve the system, we could introduce synonym matching or query expansion to retrieve more relevant products. We could also experiment with different field weights and use a more advanced ranking method, such as BM25, to improve the ordering of the results.

Finally, evaluating more queries and more retrieved documents would help us obtain a more representative assessment of the performance of the search engine.